Explainable, Uncertainty-Aware Skin Lesion Pre-Screening for Low-Resource Settings in Rwanda

Technical Report

Kayonga Elvis, BSc Software Engineering, African Leadership University

Supervisor: Dirac Murairi

October 2026

Decision-support prototype only. Not a diagnosis. Consult a qualified clinician.

Abstract

This project builds a web tool that helps a health worker decide which skin lesion photos need a specialist. It uses EfficientNet-B3 trained on HAM10000 (10,015 dermoscopy images, 7 classes). For each photo it gives the most likely class, an uncertainty score from Monte Carlo Dropout, a review flag, and a Grad-CAM heatmap. On 1,441 held-out test images, the first model reached 83.5 percent accuracy and macro F1 0.732, better than a MobileNetV2 baseline (0.673). The same model reached only 28 percent accuracy on dark-skin photos. I fine-tuned it on dark-skin photos, benign dark-skin lesions and Fitzpatrick III photos, with a test set locked before tuning. On that locked set, accuracy rose from 27.6 to 82.8 percent, but the set has only 29 photos. The deployed model catches 93.8 percent of melanomas when the review flag counts. It broke 2 of 5 rules I set before the run, and I deployed it anyway, with the reasons written down. The usability study was not run.

Introduction

Rwanda, like much of sub-Saharan Africa, has very few dermatologists. Most people with a worrying skin lesion are first seen by a health worker without dermatology training. A tool that sorts photos into "needs a specialist" and "probably fine", and says when it is unsure, could help that first step. The proposal fixed three research questions:

1. How accurately can a pretrained deep learning model classify the 7 HAM10000 lesion types, and how does a lightweight baseline compare with the proposed model?
2. Can Grad-CAM and Monte Carlo Dropout uncertainty be built into a web pre-screening system that gives non-specialists clear, uncertainty-aware support?
3. Does the system reach acceptable performance, useful explainability and good usability for a prototype meant for low-resource settings?

My own goal is that the tool works fairly for people with dark skin (Fitzpatrick V and VI). HAM10000 comes mostly from light-skinned patients and has no skin-tone labels, so I measured this gap and tried to reduce it.

Methods

Data and split

HAM10000 has several photos of the same lesion. I split by lesion, not by photo, so one lesion is never in two sets: 7,116 training, 1,458 validation and 1,441 test images. The split files are saved in the repository. About 67 percent of images are common moles, so I used a class-weighted loss and report macro scores, which treat all 7 classes equally.

Model

EfficientNet-B3 starts from ImageNet weights, with the last layer replaced by 7 outputs and a 300 by 300 input. Training used AdamW, a cosine learning rate schedule, augmentation and early stopping on validation macro F1, in Google Colab on a T4 GPU (14 epochs). MobileNetV2 was trained the same way as the baseline.

Uncertainty, review rule and heatmap

The network's feature layers run once. The last layer runs 10 times with dropout on, and the 10 outputs are averaged. The entropy of the average is the uncertainty. A photo is flagged for clinician review if entropy is at or above the 80th percentile of the validation set, or if the melanoma probability is at least 5 percent. Both thresholds come from validation data only. The low melanoma threshold is on purpose: missing a melanoma is worse than an extra referral. Grad-CAM uses the last feature block and the top class.

Dark-skin experiments

Each experiment had a pass or fail rule written before it ran. Phase 3 added PAD-UFES-20 phone photos (mostly light skin). Phase 4 trained on the 139 Fitzpatrick17k photos of skin types IV to VI that still download, measured by 5-fold cross validation. Phase 4b locked 29 of those photos and the benign lesions of 10 of 21 ISIC dark-skin patients (195 lesions) before any tuning, and used them once at the end. It added the other 215 benign dark-skin lesions with a loss that pushes probability away from the 3 cancer and pre-cancer classes, and 118 Fitzpatrick III photos. It then tried 7 changes one at a time with 4-fold cross validation on the remaining data. All dark-skin runs used a Kaggle T4 GPU.

Prototype

A FastAPI backend has thin routes, services that hold the logic, an abstract classifier and explainer, one class per photo check, and repositories for the database. A React web app shows the class, all 7 probabilities, an uncertainty meter, the review banner with its reason, and the heatmap next to the photo. There is no login screen: the browser creates an anonymous account, and every API call still needs a token. Cases are stored in SQLite, or in PostgreSQL with Docker Compose. Every result shows the disclaimer.

Implementation and verification

Scope alignment

Planned component	Status	Evidence
Upload a photo (FR-01)	Completed, plus camera	frontend/src/components/
Check format, size, resolution (FR-02)	Completed, plus blur check	backend/app/quality.py
Classify into 7 classes (FR-03)	Completed	backend/app/ml/classifier.py
Uncertainty and confidence (FR-04)	Completed	backend/app/ml/uncertainty.py
Grad-CAM heatmap (FR-05)	Completed	backend/app/ml/explainer.py
Review alert (FR-06)	Completed	backend/app/ml/review.py
Cases stored in PostgreSQL (FR-07)	Completed, checked in Docker	docs/results/docker_postgres_check.json
Documented REST API (FR-08)	Completed	Swagger UI at /docs
Authentication (FR-09)	Changed: tokens kept, no login screen	backend/app/services.py
Result within 10 seconds	Completed: about 1.5 s on CPU	docs/results/performance_*.json
Usability study, SUS of 65 or more	Not run	docs/usability/ (kit only)

Testing

The backend suite has 63 tests, all passing: 45 unit tests and 18 integration tests. Unit tests cover each photo check, the review rule, password hashing, tokens and the calibration maths, using fake models where needed. Integration tests run the whole API with the real model: register, predict, case history, every rejection path, and missing or wrong tokens. The frontend has 8 unit tests for the API client. Playwright runs 8 user journeys in Chromium, Firefox, WebKit, mobile Chrome and mobile Safari: 38 passed and 2 were skipped, because WebKit has no fake camera. GitHub Actions runs the linter and all tests on Linux and Windows for every push.

Input tests through the running app: a mole photo returned a result with no flag; a melanoma photo returned Melanoma with the review flag; a blurry photo returned 422; a text file returned 415; a call without a token returned 401. On the 29 locked dark-skin photos, the running app named 23 correctly and named or flagged 27 of 28 cancers. One basal cell carcinoma was called dermatofibroma without a flag.

A benchmark script timed the full request, heatmap included. Warm median: 1.52 s on a 2015 laptop CPU (Intel i5-6300U), 0.61 s on a server CPU and 0.17 s on an NVIDIA T4 GPU. Peak memory was 2.2 GB on the laptop. Testing led to fixes: the blur threshold went from 60 to 5.0 after it rejected 25 of 43 real photos; a warm-up at start-up cut the first prediction from 7.9 s to 1.6 s; and the uncertainty badge now uses the same threshold as the review flag.

Code quality

The backend separates routes, services, the model, photo checks, security, storage and database access into their own modules. Abstract classes are used for the classifier, the explainer and the photo checks, so each part can be replaced or faked in tests. One container builds every object at start-up. Every backend module starts with a short description of its job. The ruff linter reports no findings and runs in CI. Thresholds and class names live in one model configuration file, not in the code. The repository has 42 commits, and the README links every number to its result file.

Results

Result	Value
HAM10000 test, MobileNetV2: accuracy; macro F1; melanoma sensitivity	81.0%; 0.673; 41.8%
HAM10000 test, EfficientNet-B3 v0: accuracy; macro F1; macro AUC; melanoma sensitivity	83.5%; 0.732; 0.963; 57.5%
HAM10000 test, deployed v1: accuracy; macro F1; macro AUC; melanoma sensitivity	80.4%; 0.708; 0.955; 70.5%
Balanced accuracy (macro recall), v0; v1	74.3%; 74.6%
Review flag, v1: share flagged; accuracy not flagged; accuracy flagged	38.2%; 94.7%; 56.5%
Review flag, v1: mistakes caught; melanomas caught counting the flag	83.6%; 93.8%
Calibration of v0 with temperature 2.017: expected calibration error	0.095 to 0.021 (not used in the app)
Phase 3 (phone photos): phone accuracy; benign dark-skin false alarms	34.1% to 80.2%; 1.7% to 31.2% (not deployed)
Phase 4 (cross validation, 139 photos): dark-skin accuracy; benign dark-skin false alarms	28.1% to 81.3%; 1.7% to 38.0% (not deployed)
Phase 4b locked test, 29 photos: dark-skin accuracy (95% interval for v1)	27.6% to 82.8% (69 to 97)
Phase 4b locked test: dark-skin melanomas named; cancers named or flagged	36% to 73%; 92.9% to 100%
Phase 4b locked test, 195 benign lesions: false alarms (95% interval for v1)	0.5% to 4.1% (1.6 to 6.5)
Phase 4b rule checks passed	3 of 5

Discussion

On research question 1, EfficientNet-B3 classified the 7 lesion types moderately well and beat MobileNetV2 on every measure, most of all on melanoma (57.5 against 41.8 percent). Balanced accuracy of 74 percent is below the 88 to 93 percent I expected. Many published results let the same

lesion appear in training and test; my split does not, so my number is lower but fair. The model trained only on HAM10000 got 28 percent on dark-skin photos, so benchmark accuracy did not carry over to the people this tool is for.

On research question 2, yes. The running app shows the class, the uncertainty, the review flag with its reason, and the heatmap in about 1.5 seconds on a laptop CPU. The flag is useful: the model is 94.7 percent accurate on photos it does not flag, and the flag catches 83.6 percent of its mistakes and 93.8 percent of melanomas. That is a 6.2 percent miss rate, which meets the proposal target of under 15 percent. The cost is that 38 percent of photos go to review. The heatmaps show where the model looked, but I did not test whether this matches what a clinician would look at.

On research question 3, the answer is partly. Performance on HAM10000 is acceptable for a prototype, and the tool runs on a cheap laptop and in a phone browser. Dark skin improved a lot, from 27.6 to 82.8 percent, but on only 29 web photos with no moles, so it is not proven. Phone photos were not tested on the deployed model. Usability was not measured because the study did not run before the deadline.

The dark-skin work taught one clear lesson. More light-skin data made dark skin worse (Phase 3). Dark-skin cancer photos alone made the model think "dark skin means cancer" (Phase 4). Adding benign dark-skin lesions fixed most of that (Phase 4b). The deployed model failed 2 of my 5 rules: HAM10000 macro F1 fell by 0.024 against a limit of 0.01, and dark-skin false alarms rose by 3.6 points against a limit of 2. I deployed it because it finds far more cancers on dark skin and more melanomas on both test sets. Choosing after seeing results is what such a rule tries to stop, so I report it here. The first model is kept and can be switched back with two settings.

Limits: no clinician tested the tool, there is no ethics approval for real patients, squamous cell carcinoma is not one of the 7 classes, and the dark-skin data is small and from the web. Next steps are consented photos from Rwandan clinics with ethics approval, a test on the Diverse Dermatology Images dataset, a usability study with health workers, and Kinyarwanda and French in the interface.